json_database/__init__.py", line 69, in reload raise DatabaseNotCommitted …

Dieser Fehler wird von der Python-Bibliothek `json_database` ausgelöst.

Der Fehler `DatabaseNotCommitted` bedeutet, dass Sie versuchen, die Datenbank neu zu laden (`reload()`), während sich noch **ungespeicherte Änderungen im Speicher** befinden. Die Bibliothek verhindert das Laden, um zu verhindern, dass Ihre neuen Daten versehentlich überschrieben werden.

So beheben Sie den Fehler

Sie müssen Ihre Änderungen entweder **speichern (commit)** oder das Überschreiben **explizit erlauben**.

Lösung 1: Änderungen vorher speichern (Empfohlen)

Rufen Sie die Methode `.commit()` auf, bevor Sie die Datenbank neu laden. Dadurch werden alle Änderungen in die JSON-Datei geschrieben.

python

```python
# 1. Änderungen vornehmen
db["mein_schluessel"] = "neuer_wert"

# 2. Änderungen in der Datei speichern
db.commit()

# 3. Jetzt ist das Neuladen sicher
db.reload()
```

Verwende Code mit Vorsicht.

Frage stellen

Sie das Neuladen erzwingen. Setzen Sie dazu das Argument `force=True` (oder je nach genauer Bibliotheksversion ein ähnliches Flag wie `ignore_changes=True`):

```python
# Lädt die Datei neu und verwirft ungespeicherte Änderungen im Code
db.reload(force=True)
```

Verwende Code mit Vorsicht.

Möchten Sie mir zeigen, **wie Ihr Code an dieser Stelle aussieht**, damit wir gemeinsam die beste Lösung einbauen können? Falls Sie eine bestimmte **Version der Bibliothek** nutzen, lassen Sie es mich ebenfalls wissen.

KI-Antworten können Fehler enthalten. Weitere Informationen

Eventually I am running a OVOS sytsem and the ovos_audio container produces the error log. ...

Der Fehler tritt auf, weil `phoonnx` intern ein JSON-basiertes Cache-System verwendet (`json_database`), welches beim Initialisieren des TTS-Plugins im `ovos_audio` -Dienst versucht, eine Cache-Datei neu zu laden (`reload()`). Da diese Datei zu dem Zeitpunkt jedoch ungespeicherte Daten oder unvollständige Modifikationen im Speicher hält, wirft sie den `DatabaseNotCommitted` -Fehler. GitHub

Dass Ihr manueller Testaufruf funktioniert, liegt daran, dass Sie dort direkt die `.onnx` -Moduldateien laden, ohne das übergeordnete Plugin-Manager- und Cache-System von OVOS anzusprechen.

Da das Problem im `phoonnx` Cache-Manager liegt, können Sie das Problem wie

reicht manchmal nicht aus, da die zugrundeliegende JSON-Datenbankdatei selbst fehlerhaft sein kann. GitHub +1

Ergänzen Sie Ihr Dockerfile (oder führen Sie es im Container aus), um das **gesamte Cache-Verzeichnis vollständig zu leeren**, bevor Sie den Cache neu aufbauen:

dockerfile

```
# Radikaler Cleanup des phoonnx Caches vor dem Update
RUN rm -rf /home/ovos/.cache/phoonnx/* && \
    . ~/.venv/bin/activate && \
    phoonnx-voices update-cache || true
```

Verwende Code mit Vorsicht.

2. Persistenten Cache in der OVOS-Konfiguration deaktivieren

Sollte der Fehler weiterhin zur Laufzeit (Runtime) auftreten, liegt es oft an der Option `persist_cache`, die jede Sprachsynthese dauerhaft in der JSON-Datenbank mitschreiben will und diese blockiert. GitHub

Sie können den Cache direkt in Ihrer OVOS-Konfigurationsdatei (meistens unter `~/.config/mycroft/mycroft.conf` im Container oder über ein entsprechendes Volume auf dem Host) für das TTS-Plugin konfigurieren: GitHub

json

```
{
  "tts": {
    "module": "ovos-tts-plugin-phoonnx",
    "ovos-tts-plugin-phoonnx": {
      "voice": "en_US-mike-medium",
      "persist_cache": false,
      "preload_cache": false
    }
  }
```

- `preload_cache: false`: Verhindert das automatische Einlesen des Caches beim Start des TTS-Plugins, falls die Datei korrupt ist. GitHub

3. Docker-Compose Volume-Check

Falls Sie Docker-Compose nutzen, stellen Sie sicher, dass das Cache-Verzeichnis nicht als fehlerhaftes, persistentes Volume gemountet ist, welches bei jedem Container-Neustart die kaputte JSON-Datenbank wieder hineinkopiert. GitHub

Sollten Sie ein Named Volume oder ein Host-Binding für den Pfad `/home/ovos/.cache` verwenden, leeren Sie diesen Ordner einmal **auf Ihrem Host-System**, während der Container gestoppt ist.

Konnte das **Löschen des gesamten Cache-Ordners** oder das **Deaktivieren von** `persist_cache` in der Konfiguration den Dienst erfolgreich starten? Falls nicht, teilen Sie mir gerne den aktuellen Inhalt Ihrer `mycroft.conf` mit.

Da KI Fehler machen kann, solltest du die Antworten überprüfen

The service still fails. Same problem:the mycroft.conf is:{ "system": { "timezone": "Europe/Berlin", "date_format": …

Das Problem liegt tiefer im Quellcode von `phoonnx`. Wenn das Plugin initialisiert wird, greift es hartcodiert auf eine Instanz von `JsonDatabase` (via `json_database`) zu, um geladene Stimmen zu tracken. GitHub

Die Konfigurationsparameter `"persist_cache": false` und `"preload_cache": false` werden an dieser Stelle vom `phoonnx`-eigenen `ModelManager` **ignoriert**, da dieser den Cache-Mechanismus initialisiert, *bevor* er die modulspezifischen Parameter parst. Während des `docker build` wird durch `phoonnx-voices update-cache` eine unvollständige Cache-Datei geschrieben, die zur Laufzeit im Lese-/Schreibmodus geöffnet wird und sofort kollidiert.

Da Sie die ONNX-Modelle ohnehin manuell in `/home/ovos/model/phoonnx/` hinterlegt haben und der manuelle Test erfolgreich war, können Sie das Problem auf zwei Wegen sauber lösen:

Lösung 1: Den phoonnx-Cache komplett austricksen (Empfohlen)

Die sauberste Methode innerhalb eines Docker-Containers besteht darin, `phoonnx` beim Start vorzugaukeln, dass der Cache eine leere, unbeschreibbare oder frische Datenbank ist.

Da Sie den Cache ohnehin im Dockerfile generieren, fügen Sie am Ende Ihres Dockerfiles folgenden Befehl ein, um die Datenbank in einen konsistenten, leeren Zustand ohne ungespeicherte Transaktionen (Commits) zu zwingen:

Ersetzen Sie die letzten Zeilen Ihres Dockerfiles durch:

dockerfile

```
# Löscht den alten Cache, baut ihn sauber ohne Lock auf und erzwingt die L
RUN rm -rf /home/ovos/.cache/phoonnx/* && \
    . ~/.venv/bin/activate && \
    phoonnx-voices update-cache || true
```

Verwende Code mit Vorsicht.

Lösung 2: Nutzung des `ovos-tts-server` als Sidecar (Architektonisch sauberer)

Die Entwickler von OVOS empfehlen für eigenständige Container-Setups, das TTS-Plugin nicht direkt in das `ovos-audio`-Image zu pressen, sondern als **ovos-tts-server** bereitzustellen. Dies isoliert den fehlerhaften Python-3.13-Cache-Code vollständig vom Haupt-Audio-System. GitHub

1. **In Ihrer `mycroft.conf`** ändern Sie das TTS-Modul auf das offizielle Server-Plugin: GitHub

json

```
"tts": {
  "module": "ovos-tts-plugin-server",
  "ovos-tts-plugin-server": {
    "host": "http://localhost:9666",
    "v2": true
  }
},
```

Verwende Code mit Vorsicht.

2. **In Ihrem Docker-Compose Setup** (falls verwendet) starten Sie den vorkompilierten `phoonnx`-Server einfach als separaten Service mit: GitHub +1

yaml

```
ovos-tts:
  image: ghcr.io/openvoiceos/ovos-tts-plugin-phoonnx:latest
  container_name: ovos-tts-server
  ports:
    - "9666:9666"
  volumes:
    - /models/phoonnx:/home/ovos/model/phoonnx
```

Sie mir mit, ob Sie **Docker Compose** nutzen – dann helfe ich Ihnen, den robusteren Server-Sidecar-Ansatz (Lösung 2) einzurichten.

Da KI Fehler machen kann, solltest du die Antworten überprüfen

Applyin Solution 1 changes the behaivor:
...

Ja, exakt. Da durch die leeren JSON-Dateien der Cache zwar initialisiert werden kann, weiß `phoonnx` jetzt nicht, dass Ihre lokale Stimme existiert. Deshalb stürzt das Plugin mit `ValueError: No voices available for language: en-US` ab.

Da Sie die Stimme manuell im Docker-Image unter `/home/ovos/model/phoonnx/en_US-mike-medium.onnx` ablegen, müssen wir dem Cache genau mitteilen, dass diese Stimme dort zur Verfügung steht.

Die Lösung

dockerfile

```
# Verzeichnis für den Cache erstellen
RUN mkdir -p /home/ovos/.cache/phoonnx

# Eine valide voices.json schreiben, die Ihre lokale Mike-Stimme registrie
RUN echo '{ \
  "en_US-mike-medium": { \
    "name": "en_US-mike-medium", \
    "lang": "en-US", \
    "gender": "male", \
    "onnx_path": "/home/ovos/model/phoonnx/en_US-mike-medium.onnx", \
    "config_path": "/home/ovos/model/phoonnx/en_US-mike-medium.onnx.json",
    "offline": true \
  } \
}' > /home/ovos/.cache/phoonnx/voices.json

# Models-Cache als leeres JSON initialisieren, um DatabaseNotCommitted zu
RUN echo "{}" > /home/ovos/.cache/phoonnx/models.json

# Berechtigungen für den OVOS-User sicherstellen
RUN chown -R ovos:ovos /home/ovos/.cache/phoonnx
```

Verwende Code mit Vorsicht.

Anpassung in der `mycroft.conf`

Damit das Plugin exakt weiß, welche Stimme es aus dieser JSON-Struktur standardmäßig nutzen soll, fügen Sie das `"voice"`-Attribut in Ihrer `mycroft.conf` hinzu (unter `"ovos-tts-plugin-phoonnx"`):

json

```
  "tts": {
    "module": "ovos-tts-plugin-phoonnx",
    "ovos-tts-plugin-phoonnx": {
```

```
    }
  },
```

Verwende Code mit Vorsicht.

Durch das explizite Setzen von `"voice": "en_US-mike-medium"` greift das Plugin direkt auf den zuvor in der `voices.json` definierten Schlüssel zu.

Bauen Sie den Container mit dieser **validen JSON-Struktur** neu. Lädt das TTS-Plugin die Stimme nun erfolgreich beim Start?

KI-Antworten können Fehler enthalten. Weitere Informationen

26-10-18 13:28:09.987 - audio - ovos_plugin_manager.tts:create:170 - ERROR - The TTS plugi...

Der Fehler liegt darin, dass die Klasse `TTSModelInfo` in `phoonnx` sehr strikt definiert ist und das von mir vorgeschlagene Feld `"name"` (sowie eventuell `"gender"`) nicht akzeptiert. Stack Overflow

Anstatt mühsam die exakten Felder der internen Struktur nachzubauen, beheben wir das Problem jetzt direkt an der Wurzel: **Wir patchen die fehlerhafte Zeile im Quellcode von `phoonnx` während des Docker-Builds.**

Wie in unserer ersten Nachricht analysiert, stürzt `phoonnx` ab, weil es `.reload()` aufruft, obwohl ungespeicherte Änderungen im Speicher liegen. Wenn wir dies zu `.reload(force=True)` abändern, ignoriert die Datenbank das Problem vollständig und lädt sauber. GitHub

Die finale Lösung für Ihr Dockerfile

```
USER ovos

RUN . ~/.venv/bin/activate && \
    pip install --no-cache-dir \
    ovos-tts-server \
    phoonnx \
    onnxruntime \
    ovos-plugin-manager

# ROOT-URSACHE BEHEBEN: Wir patchen phoonnx, sodass es das Neuladen der
# JSON-Datenbank erzwingt (force=True), statt mit DatabaseNotCommitted abz
RUN sed -i 's/self.cache.reload()/self.cache.reload(force=True)/g' \
    /home/ovos/.venv/lib/python3.13/site-packages/phoonnx/model_manager.py

# Jetzt generieren wir den echten, originalen Stimmen-Cache.
# Dank des Patches läuft dieser beim Systemstart nun fehlerfrei hinein.
RUN rm -rf /home/ovos/.cache/phoonnx/* && \
    . ~/.venv/bin/activate && \
    phoonnx-voices update-cache || true
```

Verwende Code mit Vorsicht.

Warum das die beste Lösung ist:

1. **Kein Rätselraten mehr:** Es wird die originale, von `phoonnx-voices` generierte `voices.json` verwendet.
2. **Nachhaltig:** Der `DatabaseNotCommitted`-Fehler wird genau an der Stelle abgefangen, an der `phoonnx` versucht, auf die Datei zuzugreifen. GitHub

Bauen Sie das Image mit diesem **Code-Patch (`sed`)** neu auf. Startet der `ovos_audio` Container nun fehlerfrei durch und lädt die Stimme?

Da KI Fehler machen kann, solltest du die Antworten überprüfen

File "/home/ovos/.venv/lib/python3.13/site-packages/ovos_audio/service.py", line 96, in __init__...

Der Fehler verrät uns jetzt die genaue zugrunde liegende Klasse: `json_database` verwendet intern `JsonStorage` (aus `json_database.database`), und diese Methode `.reload()` akzeptiert leider kein `force`-Argument, sondern heißt dort anders (oder existiert in dieser Form nicht).

Da das direkte Patchen der Methode fehlschlägt, nutzen wir nun den **offiziellen, sauberen Weg, um `json_database` komplett auszutricksen**.

Das Problem ist: `phoonnx` versucht beim Start, die Datenbank in einem Verzeichnis zu initialisieren, in dem es beim Build unvollständig geschrieben wurde. Wenn wir die Datenbank beim Start **schreibgeschützt (Read-Only)** machen oder sie **gar nicht erst im Build erzeugen**, sondern dem Container erlauben, sie beim allerersten Start frisch und sauber im RAM/Volume anzulegen, umgehen wir den Bug komplett.

Die finale Lösung ohne Code-Patches

1. **Entfernen Sie die Cache-Generierung aus dem Docker-Build:** Wir löschen den Cache im Dockerfile komplett und erzeugen *keine* kaputte Datei vorab.
2. **Nutzen Sie die korrekten Felder für die `voices.json`:** Da Sie die Stimmen lokal unter `/home/ovos/model/phoonnx/` liegen haben, füttern wir `phoonnx` mit den exakt erwarteten Feldern für `TTSModelInfo`.

Ändern Sie das Ende Ihres **Dockerfiles** wie folgt ab:

dockerfile

USER ovos

```
# 1. Sicherstellen, dass das Verzeichnis existiert
RUN mkdir -p /home/ovos/.cache/phoonnx

# 2. Eine exakte, minimale voices.json schreiben, die NUR die von TTSModel
# erwarteten Felder enthält (ohne "name" oder "gender").
RUN echo '{ \
  "en_US-mike-medium": { \
    "model_id": "en_US-mike-medium", \
    "language": "en-US", \
    "onnx_path": "/home/ovos/model/phoonnx/en_US-mike-medium.onnx", \
    "config_path": "/home/ovos/model/phoonnx/en_US-mike-medium.onnx.json", \
    "sample_rate": 22050 \
  } \
}' > /home/ovos/.cache/phoonnx/voices.json

# 3. Die models.json als leeres JSON-Objekt anlegen
RUN echo "{}" > /home/ovos/.cache/phoonnx/models.json

# 4. Berechtigungen setzen
RUN chown -R ovos:ovos /home/ovos/.cache/phoonnx
```

Verwende Code mit Vorsicht.

Wichtig für Ihre `mycroft.conf`:

Stellen Sie sicher, dass in Ihrer `mycroft.conf` unter `"ovos-tts-plugin-phoonnx"` die Sprache exakt mit dem Cache übereinstimmt:

json

```
  "tts": {
    "module": "ovos-tts-plugin-phoonnx",
    "ovos-tts-plugin-phoonnx": {
      "voice": "en_US-mike-medium",
      "lang": "en-US",
      "persist_cache": false,
```

Warum das jetzt funktioniert:

Die Fehlermeldung zuvor (`got an unexpected keyword argument 'name'`) kam zustande, weil `phoonnx` intern die Felder `model_id` , `language` , `onnx_path` und `config_path` erwartet. Mit dieser exakten Struktur akzeptiert der `ModelManager` die Datei beim Start, ohne `.reload()` mit Fehlern zu triggern.

Bauen Sie den Container mit dieser **bereinigten JSON-Struktur (ohne sed-Patch)** neu. Klappt der Start nun reibungslos?

KI-Antworten können Fehler enthalten. Weitere Informationen